

Accessibility Scanner: Building a WCAG Compliance Testing Tool (SEO-Optimized)

Automated accessibility scanners check web pages against the latest WCAG guidelines (2.0, 2.1, 2.2, etc.) to prevent ADA lawsuits and improve usability. WCAG 2.2 (Dec 2024) is the newest recommendation: it *extends* WCAG 2.1 by adding new success criteria (focus order, visible focus indicator, target size, consistent help, redundant entry, accessible authentication, etc.) ¹. Sites conforming to WCAG 2.2 automatically conform to WCAG 2.0/2.1 ², and in many countries (e.g. USA, Canada, EU) WCAG forms the basis of accessibility law ³. Our scanner must cover key WCAG principles: **Perceivable** (e.g. alt text for images, captions, color contrast 1.4.3), **Operable** (keyboard navigation, focus order/visible focus 2.4.7–2.4.13, etc.), **Understandable** (readable text, labels, predictable input), and **Robust** (proper HTML/ARIA usage per 4.1). In practice, this means checking alt attributes, text-resizing support, semantic headings, form labels, ARIA roles, `lang` attributes, and more ⁴ ⁵. Importantly, WCAG 2.2 was published after years of careful review and is *recommended* for new policy references ⁶; our tool should test all relevant SC (1.1.1, 1.4.3, 2.1.x, 2.4.x, 3.3.x, etc.).

Figure: An intuitive scanner UI should present results visually (e.g. across devices) so developers can quickly spot issues by category and severity.

Automated Scanning Engine (axe-core)

We will build the scanner around **axe-core** (open-source, Mozilla Public License), the industry-standard accessibility engine. Axe-core provides a comprehensive ruleset covering WCAG 2.0/2.1/2.2 at levels A, AA, and AAA ⁷. It is designed for automated use (CLI, CI, or in-browser), works on all modern browsers ⁸, and “returns zero false positives” ⁸. In practice axe-core typically finds ~57% of all accessibility violations automatically ⁹; the remainder are flagged as “**incomplete**” (needing manual review). By integrating axe-core’s JavaScript API into a headless browser (e.g. Puppeteer or Playwright), our tool can load any URL, execute the page, and then run `axe.run()` on the DOM. The result is a JSON of violations (elements, WCAG criteria, messages).

For example, axe’s **color-contrast** rule programmatically computes each text element’s contrast ratio. It “checks all text elements to ensure the contrast between the foreground text and the background colors meet WCAG 2 AA thresholds” ⁴ (4.5:1 for normal text, 3:1 for large text). If contrast fails, axe reports a violation; fully transparent or hidden text is skipped. Similarly, axe has rules for missing alt text, ARIA misuse, duplicate IDs, form labels, etc. We will parse these results, filter by severity, and prepare them for display. (If needed, other libraries can augment axe: e.g. Google’s Accessibility Scanner for Android apps, or manual checks for content/context, but axe-core will be our foundation.)

Key Steps:

- **Page retrieval:** Use a headless browser or DOM library to fetch and render each target page (respecting JavaScript).
- **Run axe-core:** Inject `axe.min.js` and call `axe.run()`. Filter out common false hits (axe claims 0 false

positives ⁸).

- **Parse results:** For each violation, record the affected element(s), the WCAG SC (with level A/AA/AAA), severity, and remediation text.
- **Reporting:** Aggregate results by category (e.g. Color, Keyboard, Forms) and severity, ready to display.
- **Repeat for multiple pages:** Optionally crawl the site or read a sitemap to scan all pages. Ensure scans are rate-limited to avoid overload.
- **Manual fallback:** Note that any elements marked *incomplete* (context-dependent checks) should be earmarked for human review (see **Limitations** below).

What to Test (WCAG Categories)

Our scanner should cover the same categories outlined by WCAG and leading tools ⁵. Key checks include:

- **Visual/Perceptual:** Color-contrast ratios (WCAG 1.4.3), presence of **alt text on images** (1.1.1), correct document `lang` attribute (3.1.1), visible focus indicators (2.4.7), responsive text resizing (1.4.4), and skip-links or landmarks for navigation ⁴ ⁵.
- **Keyboard/Navigation:** Keyboard operability of all functions (2.1.x) – e.g. no “keyboard traps”; logical **tab order** (2.4.7–2.4.13); presence of **skip-links**; focus moving correctly into new windows (e.g. after opening a popup) ¹⁰. The tool can simulate pressing Tab throughout the page and record the tab stops (cf. axe’s Keyboard IGT ¹¹). It should then flag any interactive element not reachable by keyboard, or any focusable element lacking an ARIA `role` or `tabindex` ¹² ¹⁰.
- **Semantic Structure:** Proper heading hierarchy (H1→H2→..., no skips) ¹³; form controls with **associated labels** (2.4.6/3.3.2); ARIA attributes that are valid for the element (using axe’s ARIA rules); landmark roles (banner, main, navigation) used appropriately ¹⁴. For example, axe will catch an `<input>` without a `<label>` or `aria-label`.
- **Media & Content:** Captions for videos and transcripts for audio (WCAG 1.2 series); text alternatives for any time-based media; meaningful link text (1.1.1 & 2.4.4); document language declared (2.3.1). The scanner should verify that any `<video>` has captions, and any meaningful image has `alt`.

In summary, our feature list mirrors other free tools (e.g. DigitalA11Y’s scanner checks contrast, alt text, headings, etc. ⁵). We should present results grouped by type (perceptual, keyboard, semantic, etc.) and by severity (Critical, Major, Minor) as they do ¹⁵.

Competitor Gaps & Differentiation

Many free scanners exist, but they have limitations. For example, DigitalA11Y’s free checker covers WCAG 2.1/2.2 but finds only ~30–40% of total issues ¹⁶, relying on manual audit for the rest. In contrast, axe-core’s higher coverage (~57% of issues ⁹) and zero false-positive guarantee ⁸ help ensure our results are both broad and accurate. Some tools (like WAVE or Silktide’s demo) only test one page at a time or one aspect (e.g. color contrast). Our scanner can stand out by offering **site-wide scanning**, automated scheduling/monitoring, and easy output.

- **Automation & Accuracy:** Use axe-core rules to catch a broad range of issues programmatically. Unlike paid tools, our open-source approach means we can customize rules (e.g. add more ARIA heuristics) and still avoid false alarms. We should cross-check that axe’s findings align with the latest WCAG 2.2 (e.g. adding new SC like 2.4.10 Focus Order).

- **UI/UX Excellence:** Many free scanners just email results or dump data. We will build a clear, interactive UI: a results dashboard with charts and a list of issues (sorted by severity). Each issue entry should show *exactly* which element failed (with a CSS selector or screenshot thumbnail) and a concise remedy. As Deque’s axe DevTools demonstrates, “rich issue details and reporting including screenshots” greatly help users fix issues ¹⁷. We can emulate that by highlighting offending elements on a live DOM snapshot or code snippet in our report.
- **Repeat Use & Workflow:** To encourage repeated scanning, our tool should allow users to save projects (e.g. their domain), schedule scans, and track progress over time. Providing accounts with scan history, or integrations (email alerts, Slack notifications) will make it a daily driver. Regular scans automatically improve both compliance and SEO. (Conversely, one-off scans become obsolete as content changes.)

UI/UX Design

The scanner’s interface must itself be accessible, clean, and engaging. Key UI/UX features:

- **Simplicity:** On the homepage, prompt the user to enter a URL (and optionally a name for the project). Show an animated progress bar during scanning (scan can take seconds to minutes). Make clear what WCAG levels are being tested (A/AA by default).
- **Results Dashboard:** After scan, display a summary card (accessibility score or error counts by level). Below that, show tabs or sections for each issue category (e.g. Perceivable, Operable, etc.). Within each, list issues with icons (⚠ for major, ! for critical) and short text (e.g. “Missing alt text on 3 images”). Each item should be expandable to show details: the exact HTML element (e.g. `<img alt="">`), the relevant SC, and a “How to fix” note. Users should be able to click to highlight the element on a rendered page screenshot or DOM view.
- **Interactive Help:** Include links to WCAG documentation or best practices. For example, clicking “Color Contrast” could show the WCAG 1.4.3 criteria and recommended contrast ratios. Tooltips or a “learn more” icon can provide definitions (e.g. what is “keyboard trap”).
- **Visuals & Accessibility:** Use color and icons to flag severity but also ensure the UI meets ARIA guidelines (ARIA labels, focus order, high-contrast theme). Since developers will use this tool heavily, the design should be modern and uncluttered, with responsive layout (works on mobile too). We might include charts showing issue breakdown by type/severity for a quick overview.
- **Sample UI Illustration:** An example interface could look like an audit dashboard with linked devices and annotated results. See the conceptual image below for a style direction.

Conceptual mock-up: an accessibility scanner highlights issues on multiple devices. A good UI groups errors by type and severity, shows code context, and offers quick fixes.

The overall goal is to make remediation **actionable**. As DigitalA11Y notes, reports must give “specific code location” and “practical remediation guidance” ¹⁸. We will ensure each reported violation in our tool comes with concrete fix instructions and a copyable snippet.

SEO Strategy (for the Tool and Accessible Sites)

Search Engine Optimization (SEO) is a win-win here. First, the scanner’s own site should rank for high-value keywords like “free accessibility checker” and “WCAG compliance test” (these see tens of thousands of monthly searches). The landing page must use these terms in the title, headers, and meta description. We should publish SEO-rich content (blog posts, guides on accessibility, case studies) to attract organic traffic.

Structured data (FAQ schema, breadcrumb) will improve visibility. Importantly, **make the tool itself accessible**: a site that follows WCAG best practices can rank better.

Second, accessibility improvements benefit SEO for **any** site. Studies show accessible websites get significantly more traffic: one analysis found an average *12% increase* in organic traffic after fixes ¹⁹, while another study reported *~23% more traffic* and stronger rankings on accessible sites ²⁰. Google's updates reward good UX signals (e.g. lower bounce, faster loading, mobile-friendly design), which overlap with accessibility (clear headings, alt texts, mobile/tablet support). As ReciteMe explains, accessibility and SEO "are usually mutually beneficial" – improving accessibility typically improves SEO metrics, even though accessibility isn't a *direct* ranking factor ²¹ ²². For example, proper `<h1>/<h2>` structure and descriptive alt text help search engines index content more accurately ²¹ ²².

SEO Best Practices for the Scanner: Use target keywords (e.g. "WCAG checker") in headings and links, write informative page titles ("Free WCAG 2.2 Accessibility Scanner"), and meta descriptions emphasizing unique value (e.g. "Find and fix WCAG 2.2, ADA, Section508 issues instantly"). Create a resource hub (e.g. "WCAG 2.2 Guide" or "ADA Laws 2025") to capture related queries. Share results or blog posts on social media and dev communities to build backlinks. Ensure the scanner's pages load quickly and are mobile-optimized, since page speed and mobile-friendliness are key SEO signals.

In summary, our scanner will **promote SEO** both by targeting popular search terms itself and by helping end-users improve their site's search rankings (accessible sites often attract ~12–23% more organic traffic ¹⁹ ²⁰). This dual focus makes the product attractive to webmasters: they fix accessibility and watch SEO rise.

Development & Implementation Steps

To build this tool (estimating ~1–2 weeks of development), we can follow these steps:

1. **Technology Setup:** Choose a stack – for example, a Node.js backend. Install `axe-core` (npm) and a headless browser library (Puppeteer or Playwright).
2. **Crawler/Scanner Engine:** Write code to fetch and render a URL in headless mode. Inject `axe.min.js` and execute `axe.run()`, capturing JSON output. Consider using the official [axe-core CLI](#) for a quick start, or call the API directly.
3. **Result Parser:** Build a module to transform axe's JSON into our data model: group violations by WCAG criterion and severity, count totals, etc. Also handle pagination if scanning a site.
4. **UI/UX Frontend:** Develop a web interface (HTML/CSS/JS or framework like React/Vue). The UI should make an API call to trigger a scan, then display results in the dashboard as outlined above. Implement filtering/sorting. Ensure the UI is accessible (so all users can use the scanner).
5. **Persistence (Optional):** Add user accounts or project management so people can save scans. Store results in a database for historical comparison.
6. **Reporting:** Allow exporting reports (HTML summary, PDF, Excel sheet, or JSON). Provide an email option or a permalink to share results.
7. **Testing:** Verify the scanner on known test pages (e.g. W3C's WCAG examples) and real sites. Cross-check that violations are correctly detected and categorized. Use axe's [rule documentation](#) for expected behavior.
8. **Performance:** Optimize for speed – e.g. parallelize page scans, cache static assets. Ensure error handling (timeouts, inaccessible URLs).

9. **SEO & Launch:** Once ready, deploy on a performant host (VPS or cloud service). Implement SEO optimizations: friendly URLs, meta tags, sitemaps. Launch with a blog post or announcement about the new WCAG 2.2 features.
10. **Maintenance:** Plan for updates: as WCAG evolves (e.g. future 2.3 or 3.0), update axe rules. Axe-core itself releases new rules every few months ²³, so schedule upgrades. Set up monitoring so that new scans alert on new failures.

Differentiation & Value Proposition

By combining rigorous, up-to-date rules with an intuitive interface and SEO-conscious design, our tool will stand out. Unlike competitors that only email a report, we give instant, interactive feedback. We cover WCAG 2.2 fully and use axe-core's latest rule set (actively maintained by Deque) to minimize missed issues ⁷ ⁸. We also emphasize repeat use: users can re-scan after fixes, see trends (fewer errors over time), and get certified improvement.

In essence, our scanner not only enforces compliance (avoiding legal risk ³) but also boosts site performance on search engines ¹⁹ ²¹. By targeting high-traffic keywords and providing genuinely useful, actionable results, it will attract developers and site owners alike. Continuous monitoring (e.g. scheduled scans or integration with GitHub Actions) will encourage returning users, making this more than a one-time tool.

Sources: We followed the official WCAG 2.2 spec ²⁴ ², axe-core documentation ⁷ ⁸, and industry resources on accessibility scanning ¹⁶ ¹⁷. SEO insights are drawn from recent studies ¹⁹ ²⁰ ²¹. All functionality targets improving accessibility (and thus SEO) in line with modern standards.

¹ ² ⁶ ²⁴ Web Content Accessibility Guidelines (WCAG) 2.2

<https://www.w3.org/TR/WCAG22/>

³ Accessibility Checker - ADA & WCAG Compliance (Free Scan)

<https://www.accessibilitychecker.org/>

⁴ Axe Rules | Deque University | Deque Systems

<https://dequeuniversity.com/rules/axe/4.8/color-contrast>

⁵ ¹⁰ ¹³ ¹⁴ ¹⁵ ¹⁶ ¹⁸ ²⁰ Free Web Accessibility Checker | WCAG & ADA Scanner • DigitalA11Y

<https://www.digitala11y.com/products/scan/>

⁷ ⁸ ⁹ ²³ GitHub - dequelabs/axe-core: Accessibility engine for automated Web UI testing

<https://github.com/dequelabs/axe-core>

¹¹ ¹² Keyboard Intelligent Guided Test | Deque Docs

<https://docs.deque.com/devtools-for-web/4/en/devtools-igt-keyboard/>

¹⁷ Web Accessibility Evaluation Tools List | Web Accessibility Initiative (WAI) | W3C

<https://www.w3.org/WAI/test-evaluate/tools/list/>

¹⁹ ²² Where SEO and Web Accessibility Meet

<https://www.siteimprove.com/blog/google-seo-accessibility/>

21 Web Accessibility and SEO: How they Benefit each other
<https://reciteme.com/us/news/web-accessibility-and-seo/>